

Snapgram — Project Documentation

A full-stack social media web application inspired by early Instagram, built with Django.
Live at: <https://web-production-29f59.up.railway.app>

Table of Contents

1. [Project Overview](#)
 2. [Technology Stack](#)
 3. [Architecture & Project Structure](#)
 4. [Core Features & How They Work](#)
 5. [Database Design](#)
 6. [API Endpoints](#)
 7. [Deployment Architecture](#)
 8. [Interview Questions & Answers](#)
-

Project Overview

Snapgram is a full-stack social media platform inspired by early Instagram (image/video only). It allows users to: - Create accounts and authenticate securely - Share photos and videos with followers - Like, comment on, and explore posts - Follow/unfollow other users - Send direct messages and create group chats - Customize their profile with bio and profile picture

The project was built entirely with **Django** (Python web framework) and uses a **PostgreSQL** database in production (SQLite in development), with **Cloudinary** for media storage and **Railway** for cloud deployment.

Technology Stack

Backend

TECHNOLOGY	VERSION	PURPOSE
Python	3.11	Core programming language
Django	4.2	Web framework — handles routing, ORM, auth, templating
Gunicorn	25.1	WSGI production server
PostgreSQL	16	Production relational database
SQLite	built-in	Local development database

Frontend

TECHNOLOGY	PURPOSE
HTML5	Page structure & semantic markup
CSS3	Styling — CSS Variables, Grid, Flexbox, animations
Vanilla JavaScript	AJAX requests, real-time polling, DOM manipulation
Font Awesome 6	Icons throughout the UI
Google Fonts (Inter)	Typography

Storage & Infrastructure

TECHNOLOGY	PURPOSE
Cloudinary	Persistent cloud media storage (images, videos)
WhiteNoise	Serves static files (CSS, JS) efficiently in production
Railway	Cloud deployment platform (PaaS)
GitHub	Version control & CI/CD trigger for Railway

Key Python Packages

PACKAGE	PURPOSE
<code>django</code>	Core web framework
<code>Pillow</code>	Image processing for uploaded files
<code>cloudinary</code>	Cloudinary Python SDK
<code>django-cloudinary-storage</code>	Django storage backend for Cloudinary
<code>dj-database-url</code>	Parse DATABASE_URL env var for PostgreSQL
<code>psycopg2-binary</code>	PostgreSQL adapter for Python
<code>whitenoise</code>	Static file serving middleware
<code>gunicorn</code>	Production WSGI server



Architecture & Project Structure

```
snapgram/                                ← Django project root
|
├── snapgram/                            ← Project configuration
|   ├── settings.py                     ← All settings (DB, Cloudinary, WhiteNoise, auth)
|   ├── urls.py                         ← Root URL configuration
|   └── wsgi.py                         ← WSGI entry point for Gunicorn
|
├── core/                               ← Main app (users, posts, likes, follows)
|   ├── models.py                      ← Profile, Post, Like, Follow, Comment models
|   ├── views.py                       ← All view functions (feed, profile, post CRUD)
|   ├── urls.py                        ← URL patterns for core app
|   ├── forms.py                       ← Django forms (signup, login, post, profile)
|   ├── admin.py                       ← Django admin registration
|   ├── signals.py                     ← Auto-create Profile on User creation
|   ├── context_processors.py           ← Inject unread message count globally
|   └── management/commands/           ← Custom management commands
|       ├── create_admin.py             ← Auto-create superuser on deploy
|       ├── wipe_data.py                ← Wipe all data (dev/reset utility)
|       └── setup_fresh.py              ← Fresh setup for new deployments
```

```

|
├─ chat/                                ← Messaging app (DMs + group chats)
|   ├── models.py                       ← Message, GroupChat, GroupMember, GroupMessage
|   ├── views.py                        ← Inbox, conversation, group chat views
|   └─ urls.py                          ← URL patterns for chat app
|
├─ templates/                           ← All HTML templates
|   ├── base.html                       ← Base template (sidebar, navbar, dark mode)
|   ├── auth/
|   |   ├── login.html
|   |   └─ signup.html
|   ├── core/
|   |   ├── feed.html                   ← Home feed with posts
|   |   ├── profile.html                ← User profile page
|   |   ├── edit_profile.html           ← Edit bio/avatar
|   |   ├── create_post.html            ← Upload image/video post
|   |   ├── post_detail.html            ← Single post with comments
|   |   ├── explore.html                ← Browse all posts grid
|   |   └─ search.html                  ← Search users
|   └─ chat/
|       ├── inbox.html                  ← All conversations
|       ├── conversation.html           ← DM chat window
|       ├── group_conversation.html     ← Group chat window
|       └─ create_group.html            ← Create new group chat
|
├─ static/
|   └─ css/style.css                    ← Full CSS (light + dark mode, responsive)
|
├─ Procfile                             ← Railway/Heroku start command
├─ railway.json                          ← Railway build + deploy configuration
├─ runtime.txt                           ← Python version specification
├─ requirements.txt                       ← All Python dependencies
└─ .gitignore                            ← Files excluded from version control

```

Design Pattern

Snapgram uses Django's **MVT (Model-View-Template)** pattern: - **Model** — defines data structure and database schema - **View** — handles business logic and HTTP requests/responses - **Template** — renders HTML with dynamic data using Django's template language

Core Features & How They Work

1. Authentication System

Technology used: Django's built-in `django.contrib.auth`

Django provides a complete authentication system out of the box. We extended it by: -
Creating a custom `SignUpForm` using `UserCreationForm` with extra fields (email, first/last name) - Using Django's `authenticate()` and `login()` functions for session-based auth -
Protecting views with `@login_required` decorator - Using `LOGIN_URL`, `LOGIN_REDIRECT_URL` settings for redirects

```
# views.py - login logic
user = authenticate(request, username=username, password=password)
if user:
    login(request, user)
    return redirect('feed')
```

Why it helps: Django handles password hashing (PBKDF2), session management, and CSRF protection automatically — no need to build security from scratch.

2. User Profiles

Technology used: Django `OneToOneField`, Django Signals, Cloudinary

Each `User` (Django's built-in) has a linked `Profile` model with bio and profile picture. We use **Django Signals** to automatically create a Profile whenever a new User is registered:

```
# signals.py
@receiver(post_save, sender=User)
def create_profile(sender, instance, created, **kwargs):
    if created:
        Profile.objects.get_or_create(user=instance)
```

Profile pictures are stored on **Cloudinary** in production, so they persist across server restarts.

3. Posts (Images & Videos)

Technology used: Django `FileField` / `ImageField`, Cloudinary, Pillow

Users can upload images or videos. The `Post` model uses: - `ImageField` for photos (processed by **Pillow**) - `FileField` for videos

In production, both are stored on **Cloudinary** via `django-cloudinary-storage`. The upload form uses `enctype="multipart/form-data"` and JavaScript provides a drag-and-drop preview before submission.

4. ❤️ Likes System

Technology used: Django ORM, AJAX (Fetch API), JsonResponse

Likes are implemented with a `Like` model with a `unique_together` constraint preventing duplicate likes:

```
class Like(models.Model):
    user = models.ForeignKey(User, on_delete=models.CASCADE)
    post = models.ForeignKey(Post, on_delete=models.CASCADE)
    class Meta:
        unique_together = ('user', 'post')
```

The like/unlike action uses **AJAX** (JavaScript `fetch()`) so the page doesn't reload. The view returns a `JsonResponse` with the new like count:

```
def like_post(request, post_id):
    like, created = Like.objects.get_or_create(user=request.user, post=post)
    if not created:
        like.delete()
    return JsonResponse({'liked': created, 'likes_count': post.likes_count()})
```

5. 👤 Follow System

Technology used: Django ORM, AJAX, `unique_together`

The `Follow` model creates a many-to-many relationship between users:

```
class Follow(models.Model):
    follower = models.ForeignKey(User, related_name='following_set')
    following = models.ForeignKey(User, related_name='followers_set')
```

```
class Meta:
    unique_together = ('follower', 'following')
```

The feed shows posts from followed users using Django ORM's `Q` objects for complex queries:

```
following_users = Follow.objects.filter(follower=request.user).values_list('following', flat=True)
posts = Post.objects.filter(Q(user__in=following_users) | Q(user=request.user))
```

6. Direct Messaging

Technology used: Django ORM, AJAX polling, JsonResponse

DMs use a simple `Message` model with sender/receiver. Real-time feel is achieved through **JavaScript polling** — every 3 seconds the frontend fetches new messages:

```
// Polls for new messages every 3 seconds
function poll() {
    fetch(`/chat/dm/${username}/messages/?after=${lastId}`)
    .then(r => r.json())
    .then(data => {
        data.messages.forEach(m => {
            if (m.id > lastId) { lastId = m.id; appendMsg(m); }
        });
    });
}
setInterval(poll, 3000);
```

Why polling instead of WebSockets? WebSockets require Django Channels and Redis — complex to set up and expensive to host. Polling works well for a small app and is simpler to deploy.

7. Group Chats

Technology used: Django ManyToManyField, through model, AJAX

Group chats use three models: - `GroupChat` — the group itself (name, avatar, creator) - `GroupMember` — through model linking users to groups with roles (admin/member) - `GroupMessage` — messages within a group, with `read_by` M2M field for read receipts

```
class GroupChat(models.Model):
    members = models.ManyToManyField(User, through='GroupMember')

class GroupMember(models.Model):
    group = models.ForeignKey(GroupChat)
    user = models.ForeignKey(User)
    role = models.CharField(choices=[('admin', 'Admin'), ('member', 'Member')])
```

8. 🌙 Dark Mode

Technology used: CSS Variables, JavaScript `localStorage`

Dark mode uses CSS custom properties (`--bg` , `--surface` , `--text` etc.) defined on `:root` and overridden under `html.dark` . JavaScript toggles the class and saves preference to `localStorage` :

```
function toggleDark() {
    const isDark = !document.documentElement.classList.contains('dark');
    localStorage.setItem('snapgram-dark', isDark ? '1' : '0');
    document.documentElement.classList.toggle('dark', isDark);
}
```

A flash-prevention script in `<head>` applies the dark class before the page renders, preventing a white flash.

9. 📱 Responsive Design

Technology used: CSS Grid, Flexbox, Media Queries, CSS Variables

The layout adapts across screen sizes: - **Desktop (>935px)**: Fixed sidebar navigation, multi-column feed layout - **Mobile (≤935px)**: Top bar + bottom tab navigation, full-width single column

The sidebar becomes a hamburger drawer on mobile, sliding in with a CSS transform animation.

Database Design

Entity Relationship Overview

```
User (Django built-in)
├─ Profile (OneToOne) – bio, profile_pic
├─ Post (ForeignKey) – image/video, caption
│   └─ Like (ForeignKey) – user + post
│   └─ Comment (ForeignKey) – user + post + text
├─ Follow (ForeignKey×2) – follower + following
├─ Message (ForeignKey×2) – sender + receiver + content
└─ GroupMember (ForeignKey×2) – user + group + role
    └─ GroupChat
        └─ GroupMessage (ForeignKey×2) – sender + group + content
            └─ read_by (ManyToMany → User)
```

Key Design Decisions

- `unique_together` on Like and Follow prevents duplicates at the database level
- `related_name` on Follow allows querying `user.following_set` and `user.followers_set`
- `on_delete=CASCADE` ensures no orphaned data when users/posts are deleted
- `ManyToManyField` with `through` on GroupChat gives full control over membership metadata (role, joined_at)

API Endpoints

METHOD	URL	AUTH	DESCRIPTION
GET/ POST	<code>/signup/</code>	No	User registration
GET/ POST	<code>/login/</code>	No	User login
GET	<code>/logout/</code>	Yes	User logout
GET	<code>/</code>	Yes	Home feed

METHOD	URL	AUTH	DESCRIPTION
GET	/explore/	Yes	Browse all posts
GET	/search/?q=	Yes	Search users
GET/ POST	/post/create/	Yes	Create new post
GET	/post/<id>/	Yes	Post detail
POST	/post/<id>/like/	Yes	Toggle like (AJAX)
POST	/post/<id>/comment/	Yes	Add comment
POST	/post/<id>/delete/	Yes	Delete post
GET	/profile/<username>/	Yes	User profile
POST	/profile/<username>/ follow/	Yes	Toggle follow (AJAX)
GET/ POST	/settings/profile/	Yes	Edit profile
GET	/chat/	Yes	Message inbox
GET/ POST	/chat/dm/<username>/	Yes	DM conversation
POST	/chat/dm/<username>/send/	Yes	Send DM (AJAX)
GET	/chat/dm/<username>/ messages/	Yes	Poll new DMs (AJAX)
GET/ POST	/chat/group/create/	Yes	Create group
GET	/chat/group/<id>/	Yes	Group conversation
POST	/chat/group/<id>/send/	Yes	Send group message (AJAX)
GET	/chat/group/<id>/messages/	Yes	Poll group messages (AJAX)

METHOD	URL	AUTH	DESCRIPTION
POST	<code>/chat/group/<id>/add-member/</code>	Yes (admin)	Add member
POST	<code>/chat/group/<id>/remove-member/</code>	Yes (admin)	Remove member
POST	<code>/chat/group/<id>/leave/</code>	Yes	Leave group

Deployment Architecture

```

Developer Machine
|
|  git push
|
▼
GitHub Repo (kiinshuk/sg)
|
|  webhook trigger
|
▼
Railway Platform
|
├─ Nixpacks (auto build)
|   └─ pip install -r requirements.txt
|   └─ python manage.py collectstatic
|
├─ Start Command (Procfile)
|   └─ python manage.py migrate
|   └─ python manage.py create_admin
|   └─ gunicorn snapgram.wsgi --bind 0.0.0.0:$PORT
|
├─ PostgreSQL (Railway managed DB)
|   └─ Connected via DATABASE_URL env var
|
└─ Environment Variables
    └─ SECRET_KEY
    └─ DEBUG=False
    └─ ALLOWED_HOSTS
    └─ CSRF_TRUSTED_ORIGINS
    └─ DATABASE_URL (auto-set by Railway PostgreSQL)
  
```

```
|— CLOUDINARY_CLOUD_NAME
|— CLOUDINARY_API_KEY
|— CLOUDINARY_API_SECRET
```

Media Files → Cloudinary CDN (persistent, global delivery)

Static Files → WhiteNoise (served directly by Gunicorn)

Why Railway?

- Auto-detects Python/Django apps
- Free PostgreSQL addon
- Auto-deploys on every GitHub push
- Environment variable management
- No server management needed

Why Cloudinary?

- Railway's filesystem is **ephemeral** — files are deleted on every redeploy
- Cloudinary provides permanent cloud storage with a generous free tier (25GB)
- Global CDN for fast image/video delivery worldwide
- `django-cloudinary-storage` integrates seamlessly as a Django storage backend

Interview Questions & Answers

Q1: "Tell me about this project."

Answer: "Snapgram is a full-stack social media web application I built using Django and Python. It's inspired by early Instagram — focused purely on photo and video sharing.

The app includes complete user authentication, a follow system, a personalized feed, post likes and comments, direct messaging between users, and group chats. I built it as a monolithic Django application using the MVT (Model-View-Template) pattern.

For the frontend, I used HTML, CSS with CSS Variables for theming including a dark mode, and vanilla JavaScript for AJAX interactions like liking posts without page reloads and real-time message polling.

I deployed it on Railway with a PostgreSQL database and Cloudinary for media storage, and it's fully live and accessible online."

Q2: "Why Django? Why not Flask or FastAPI?"

Answer: "Django was the right choice for this project for several reasons:

First, it comes with **batteries included** — built-in user authentication, admin panel, ORM, form handling, and CSRF protection. Flask would have required me to bolt on separate libraries for each of these.

Second, Django's **ORM** made database operations very clean and Pythonic. Writing `Post.objects.filter(user__in=following_users)` is much more readable than raw SQL.

Third, Django's **admin panel** gave me a free management interface for the database without writing any extra code.

FastAPI would be good if I was building a pure REST API with a separate React frontend — but since I used server-side rendering with Django templates, Django was the natural choice."

Q3: "How did you implement real-time messaging without WebSockets?"

Answer: "I used **AJAX polling** — every 3 seconds, the frontend JavaScript makes a fetch request to the server asking for new messages since the last message ID it received:

```
fetch(`/chat/dm/${username}/messages/?after=${lastId}`)
```

The server queries the database for any messages newer than that ID and returns them as JSON. New messages are then appended to the DOM without a page reload.

This approach is simpler than WebSockets which would require Django Channels and a Redis broker — both of which add complexity and cost. For a small social app, polling every 3 seconds provides a good enough real-time experience. If I were scaling this, I'd upgrade to Django Channels with WebSockets."

Q4: "How does the follow system and feed work?"

Answer: "The Follow model has two ForeignKeys to the User model — `follower` and `following` — with a `unique_together` constraint to prevent duplicate follows.

The feed query uses Django's `Q` objects to combine conditions — it fetches posts from users that the logged-in user follows, plus their own posts:

```
following_users = Follow.objects.filter(
    follower=request.user
).values_list('following', flat=True)
```

```
posts = Post.objects.filter(
    Q(user__in=following_users) | Q(user=request.user)
).order_by('-created_at')
```

`values_list('following', flat=True)` returns a flat list of user IDs which is very efficient — it's a single database query that generates a subquery."

Q5: "How did you handle media file storage in production?"

Answer: "This was one of the more interesting challenges. Railway's filesystem is **ephemeral** — any files uploaded to the server are deleted whenever the container restarts or redeploys.

My solution was to use **Cloudinary** as the storage backend. I used the `django-cloudinary-storage` package which implements Django's file storage interface — so from Django's perspective, it's just saving a file normally, but under the hood it uploads to Cloudinary's servers.

I configured it conditionally in settings — if `CLOUDINARY_CLOUD_NAME` is set as an environment variable, use Cloudinary storage; otherwise fall back to the local filesystem for development:

```
if CLOUDINARY_CLOUD_NAME:
    DEFAULT_FILE_STORAGE = 'cloudinary_storage.storage.MediaCloudinaryStorage'
```

This means the same codebase works locally and in production without any changes."

Q6: "How did you implement dark mode?"

Answer: "Dark mode is implemented using **CSS Custom Properties** (variables) and a JavaScript class toggle on the `<html>` element.

I defined two sets of variables — the default light theme on `:root` and a dark override under `html.dark`:

```
:root { --bg: #f5f5f5; --text: #0f0f0f; }
html.dark { --bg: #0a0a0a; --text: #f5f5f5; }
```

Every component references these variables, so toggling the class instantly changes the entire theme.

The user's preference is saved to `localStorage` and re-applied on the next page load. To prevent a white flash before JavaScript runs, I added an inline script in the `<head>` that reads `localStorage` and adds the dark class synchronously before the page renders."

Q7: "How is the app secured?"

Answer: "Several layers of security:

1. **CSRF Protection** — Django's built-in CSRF middleware protects all POST requests. Every form includes a `{% csrf_token %}` and AJAX requests include the token in headers.
 2. **Authentication** — All sensitive views are protected with Django's `@login_required` decorator, which redirects unauthenticated users to the login page.
 3. **Password hashing** — Django uses PBKDF2 with SHA256 by default — passwords are never stored in plain text.
 4. **SQL injection prevention** — Django's ORM parameterizes all queries, so SQL injection is not possible through normal usage.
 5. **Production security headers** — In production (DEBUG=False), I enable HSTS, secure cookies, and SSL redirect.
 6. **Environment variables** — Sensitive keys (SECRET_KEY, database URL, Cloudinary credentials) are never hardcoded — they're injected as environment variables on Railway."
-

Q8: "What would you improve if you had more time?"

Answer: "Several things:

1. **Real-time with WebSockets** — Replace the 3-second polling with Django Channels and WebSockets for truly instant messaging.
2. **Stories feature** — 24-hour disappearing posts like Instagram Stories, using a `expires_at` timestamp field.
3. **Notifications system** — Real-time notifications for likes, follows, and comments.
4. **Image optimization** — Compress and resize images before uploading to Cloudinary to save bandwidth.
5. **REST API + React frontend** — Separate the backend into a REST API using Django REST Framework and build the frontend in React for better performance and a more app-like experience.

6. **Caching** — Add Redis caching for the feed query which can become expensive as the number of posts grows.
 7. **Testing** — Write unit tests and integration tests using Django's test framework."
-

Q9: "What was the hardest part of building this?"

Answer: "The trickiest part was handling **media file persistence on Railway**. Railway uses ephemeral containers — any file uploaded to the server gets wiped whenever the container restarts. I initially didn't realize this and couldn't understand why profile pictures kept disappearing after deploys.

The solution was integrating Cloudinary as Django's `DEFAULT_FILE_STORAGE` backend, so all uploaded files go to Cloudinary's permanent storage instead of the local filesystem. This taught me an important lesson about **the difference between stateless application servers and stateful storage** — in modern cloud deployments, you should never rely on the local filesystem for anything that needs to persist."

Q10: "How does the group chat work technically?"

Answer: "Group chat uses three models:

- `GroupChat` — stores the group name, description, avatar, and creator
- `GroupMember` — a through model for the many-to-many relationship between users and groups, adding a `role` field (admin or member)
- `GroupMessage` — messages within the group, with a `read_by` ManyToMany field to track who has read each message

Using a `through` model instead of a plain `ManyToManyField` was important because I needed to store metadata about each membership — specifically the role. A plain M2M doesn't allow that.

Read receipts work by calling `msg.read_by.add(request.user)` whenever a user loads or polls messages. Unread count for the inbox badge is calculated as messages not sent by the current user and not in their `read_by` set."

Documentation written for Snapgram — Built with Django, deployed on Railway GitHub:
<https://github.com/kiinshuk/sg> Live: <https://web-production-29f59.up.railway.app>